

Référentiel de Données

Projet Décisionnel CHU - Livrable 1

CESI - École d'Ingénieurs

PGE A3 FISE - Bloc Stockage & Big Data 2025/2026

Groupe 2 : Seyni, Omar, Abdoulaye, Thierno

Résumé exécutif

Contexte

Le groupe CHU (Cloud Healthcare Unit) a pour mission de mettre en place un entrepôt de données décisionnel afin d'exploiter les volumes importants de données hétérogènes générés par ses systèmes de gestion des soins et ses systèmes FTP. Ce document constitue le **Livrable 1** du projet : le référentiel de données.

Ce rapport décrit l'ensemble des choix de conception retenus pour la solution Big Data du groupe CHU, à savoir :

- L'architecture technique de l'entrepôt de données (pattern médaillon) et la justification des choix de la stack technologique (Docker, PostgreSQL, Python).
- Le modèle dimensionnel en constellation modélisant les activités cliniques et administratives.
- Les pipelines ETL d'alimentation du schéma décisionnel.
- Les mesures de conformité RGPD appliquées aux données de santé.

Attention Conformité RGPD - Article 9

Les données traitées dans ce projet sont des **données de santé**. La pseudonymisation des identifiants patients (hachage SHA-256) est obligatoire avant tout chargement dans l'entrepôt. Cette contrainte architecturale régit l'intégralité des flux présentés.

Table des matières

Table des matières	2
1 Introduction générale	5
1.1 Contexte du projet CHU	5
1.2 Objectifs du système décisionnel Big Data	5
1.3 Périmètre fonctionnel et données disponibles	6
1.4 Positionnement du Livrable 1 dans le projet global	6
2 Architecture de l'entrepôt de données (modèle 3 zones)	7
2.1 Principes fondamentaux et choix stratégiques	7
2.1.1 Alignement théorique	7
2.1.2 Étude comparative et diagnostic de l'infrastructure	8
2.1.3 Scénarios d'organisation des données étudiés	9
2.2 Description des composants techniques conteneurisés	10
2.2.1 Zone Bronze : stockage brut des sources	10
2.2.2 Zone Silver : traitement, nettoyage et orchestration	10
2.2.3 Zone Gold : stockage analytique (OLAP)	10
2.2.4 Couche restitution (exposition)	11
2.3 Flux de données et cinématique des traitements	11
2.3.1 Enchaînement des étapes d'alimentation	11
2.3.2 Stratégie d'actualisation et points de contrôle	11
2.4 Schéma d'architecture technique	12
2.5 Synthèse de l'architecture retenue	12
2.5.1 Forces, limites et scalabilité de la solution	12
2.5.2 Évolutivité de l'architecture	13
3 Modélisation dimensionnelle (Zone Gold)	15
3.1 Expression des besoins analytiques et indicateurs métiers	15
3.1.1 Indicateurs liés aux consultations et hospitalisations	15
3.1.2 Indicateurs liés aux décès	15
3.1.3 Indicateurs liés à la satisfaction des usagers	16
3.1.4 Synthèse des mesures calculées (couche restitution)	16
3.2 Choix du modèle décisionnel	17
3.2.1 Justification d'un schéma en constellation	17
3.2.2 Granularité des faits et niveaux d'agrégation	19

3.3	Spécification des tables de faits	20
3.3.1	Table de faits FAIT_HOSPITALISATION	20
3.3.2	Table de faits FAIT_CONSULTATION	21
3.3.3	Table de faits FAIT_DECES	22
3.3.4	Table de faits FAIT_SATISFACTION	23
3.4	Spécification des tables de dimensions	25
3.5	Intégration, intégrité et couverture du modèle	27
3.5.1	Clés primaires, clés étrangères et intégrité référentielle	27
3.5.2	Matrice de couverture des indicateurs métiers	27
4	Scripts ETL Python d'alimentation (Zone Silver)	28
4.1	Principes généraux de développement des pipelines	28
4.1.1	Structure type d'un script ETL	28
4.1.2	Règles communes de transformation	29
4.1.3	Module centralisé <code>etl/utils.py</code>	29
4.2	Pipeline d'alimentation des données médico-administratives	30
4.2.1	Périmètre et source	30
4.2.2	Logique de transformation	30
4.3	Pipeline d'intégration du référentiel des établissements	31
4.3.1	Périmètre et source	31
4.3.2	Logique de transformation	31
4.4	Pipeline de traitement des données de satisfaction	32
4.4.1	Périmètre et source	32
4.4.2	Logique de transformation	32
4.5	Pipeline de traitement des registres de décès	34
4.5.1	Périmètre et source	34
4.5.2	Logique de transformation	34
5	Conformité RGPD et sécurité des données	37
5.1	Cadre réglementaire et enjeux	37
5.1.1	L'article 9 du RGPD et nos sources de données	37
5.1.2	Le risque de réidentification indirecte	38
5.2	Sécurité de l'infrastructure Docker	40
5.2.1	Isolation réseau par zone	40
5.2.2	Séparation des privilèges en base de données	40
5.3	Techniques de protection dans les jobs ETL	41
5.3.1	Pseudonymisation HMAC-SHA256 tronqué	41
5.3.2	Aucune donnée nominative au-delà de la zone Bronze	41
5.3.3	Gestion des secrets via fichiers <code>.env</code> et coffre-fort	42
5.4	Gouvernance des données	43

5.4.1	Registre des traitements (Article 30)	43
5.4.2	Politique Git : données réelles interdites dans le dépôt	43
5.4.3	Analyse d'impact (DPIA) et droits des patients	44
5.4.4	Synthèse opérationnelle des mesures en place	45
6	Synthèse et perspectives	46
6.1	Bilan du référentiel de données établi	46
6.2	Limites de la solution proposée	46
6.3	Transition vers les étapes suivantes	47
	Glossaire	49
	Table des figures	52
	Table des figures	52
	Liste des tableaux	53
	Liste des tableaux	53
A	Mapping technologique détaillé	54
B	Registre complet des tables du schéma datawarehouse	55
C	Politique de qualité des données	56

1. Introduction générale

1.1 Contexte du projet CHU

La transformation numérique du secteur hospitalier constitue l'un des enjeux majeurs de la décennie, tant sur le plan organisationnel que technologique. Les établissements de santé produisent quotidiennement des volumes considérables de données hétérogènes, issues de systèmes d'information cliniques, administratifs et logistiques souvent cloisonnés. Cette fragmentation des données représente un frein à la prise de décision éclairée et à l'optimisation des parcours de soins.

Le présent projet s'inscrit dans le cadre d'une initiative de valorisation des données d'un groupe hospitalier universitaire (CHU). L'objectif est de concevoir un système d'information décisionnel à partir de données médico-administratives réelles ou simulées, en mobilisant des outils adaptés au traitement analytique de données massives. Ce projet est conduit dans un cadre pédagogique de première année de cycle ingénieur informatique au sein de l'école CESI, et constitue le premier livrable d'un projet décisionnel structuré en trois phases successives.

1.2 Objectifs du système décisionnel Big Data

Le système décisionnel conçu dans le cadre de ce projet vise à répondre à des besoins analytiques précis, formulés par des profils décisionnels tels que les directeurs d'établissements, les médecins coordinateurs et les responsables qualité.

Ces besoins se déclinent en huit indicateurs clés, constituant le périmètre fonctionnel du système :

N°	Domaine	Indicateur
1	Consultations	Nombre de consultations par période et par établissement
2	Consultations	Répartition des consultations par spécialité médicale
3	Hospitalisations	Durée moyenne de séjour par diagnostic et par établissement
4	Hospitalisations	Taux d'occupation des lits par établissement et par période
5	Décès	Taux de mortalité par région et par pathologie
6	Décès	Evolution temporelle des décès par cause et par territoire
7	Satisfaction	Score moyen de satisfaction par établissement
8	Satisfaction	Distribution des notes de satisfaction par dimension évaluée

Table 1.1 – Indicateurs décisionnels du système CHU

1.3 Périmètre fonctionnel et données disponibles

Les sources de données mises à disposition dans le cadre de ce projet sont les suivantes :

Source	Type	Format	Contenu
PostgreSQL	Structuré	BDD	Données médico-administratives des patients, consultations, etc.
Etablissements	Structuré	CSV (;)	Référentiel national des établissements (FINESS).
Satisfaction	Semi-structuré	Fichiers plats	Notes et verbatims de satisfaction patient (2020).
Décès	Semi-structuré	Fichiers plats	Répertoire national des décès, agrégé par région (2019).

Table 1.2 – Sources de données disponibles

1.4 Positionnement du Livrable 1 dans le projet global

Ce projet est structuré en trois livrables successifs. Ce premier document, le **Livrable 1**, pose les fondations conceptuelles et techniques. Il formalise le schéma en constellation, documente les flux ETL, justifie l'architecture technique retenue et statue sur la conformité RGPD. Les livrables suivants s'appuieront sur ce socle pour l'implémentation physique, l'optimisation (Livrable 2) et la restitution visuelle (Livrable 3).

2. Architecture de l'entrepôt de données (modèle 3 zones)

Ce chapitre décrit l'architecture technique retenue pour le projet décisionnel du CHU. Il relie les principes théoriques de la modélisation décisionnelle aux choix concrets de stack logicielle, de conteneurisation et d'orchestration des traitements.

2.1 Principes fondamentaux et choix stratégiques

2.1.1 Alignement théorique

Implémentation du pattern de Ralph Kimball

Le modèle de Ralph Kimball sert de fondation à la partie décisionnelle de l'entrepôt. Il repose sur un schéma en étoile constitué d'une table de faits centrale qui porte les mesures quantitatives, entourée de dimensions dénormalisées décrivant le contexte des événements.

Dans le cas du CHU, les principes suivants sont appliqués :

- Une table de faits centrale, **FACT_ACTIVITE_MEDICALE**, qui regroupe les mesures liées aux consultations, hospitalisations, décès et satisfaction.
- Des dimensions partagées (temps, patient, établissement, diagnostic, professionnel) qui assurent la cohérence des analyses croisées.
- L'utilisation de clés de substitution et de mécanismes SCD de type 2 pour historiser les changements structurants sans complexifier les requêtes.

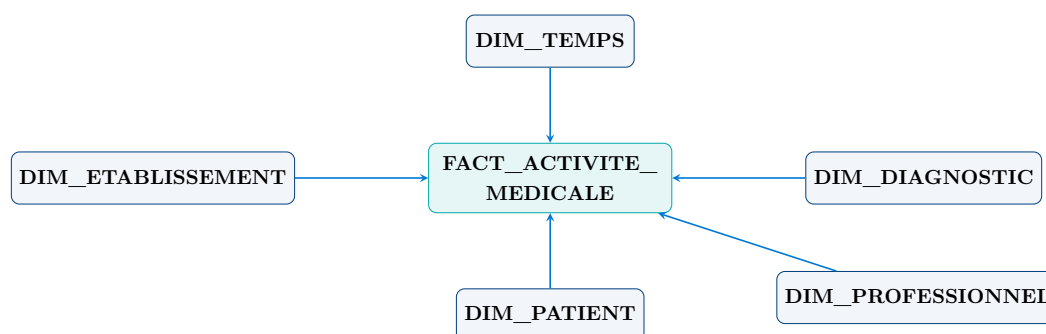


Figure 2.1 – Principe du schéma en étoile pour l'activité médicale du CHU

Cette vue conceptuelle est détaillée au chapitre 3 avec la déclinaison en constellation et les différentes tables de faits spécialisées.

Application industrielle moderne : architecture en médaille

L'architecture en médaille organise le cycle de vie de la donnée en trois zones successives. Elle complète le modèle de Kimball en structurant les étapes d'ingestion, d'intégration et d'exposition analytique.

- Zone Bronze : ingestion brute, stockage immuable et horodaté des sources.
- Zone Silver : intégration, nettoyage, normalisation des référentiels et pseudonymisation.
- Zone Gold : stockage analytique et modèle décisionnel prêt à être interrogé par la couche BI.

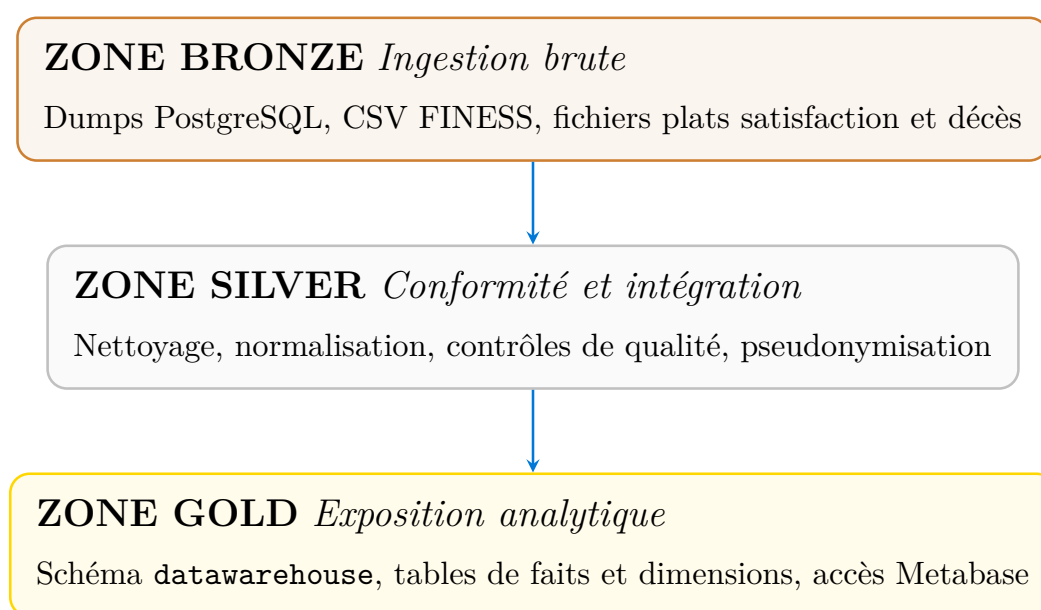


Figure 2.2 – Architecture en médaille Bronze, Silver, Gold pour le CHU

2.1.2 Étude comparative et diagnostic de l'infrastructure

Analyse de la stack VM initiale

La stack initialement envisagée reposait sur une machine virtuelle unique embarquant Hadoop, HDFS, Hive, Talend Open Studio et un outil de restitution. Cette approche souffrait de plusieurs limites dans le contexte du projet :

- Empilement de services lourds sur une seule VM, entraînant une consommation de RAM statique élevée.
- Latence d'exécution significative des traitements MapReduce en environnement local pour une volumétrie qui reste modérée.
- Courbe d'apprentissage importante pour des outils dont l'usage ne serait pas justifié à long terme sur un périmètre régional.

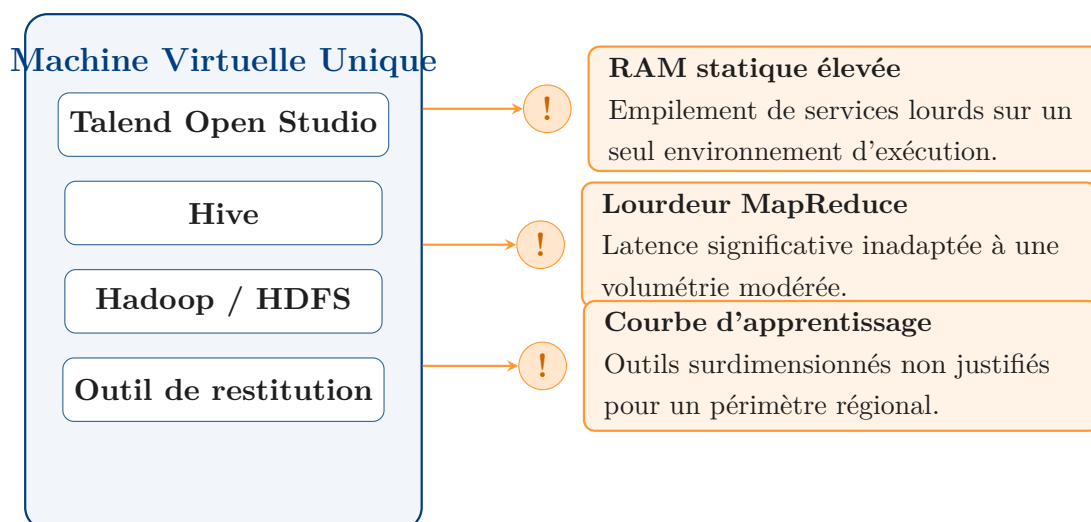


Figure 2.3 – Diagnostic et limites de l'architecture basée sur une Machine Virtuelle unique

Justification de la stack Docker, Python, PostgreSQL

La stack retenue repose sur Docker, Python et PostgreSQL, complétés par Metabase pour la restitution. Ce choix est motivé par les points suivants :

- Conteneurs plus légers qu'une VM monolithique, avec une consommation en ressources largement inférieure pour une même charge de travail.
- Flexibilité des pipelines Python (pandas, SQLAlchemy) pour intégrer des données structurées et semi-structurées sans dépendre d'un outil graphique comme Talend.
- Déploiement reproductible via Infrastructure as Code grâce au fichier `docker-compose.yml`.
- PostgreSQL utilisé à la fois comme base opérationnelle répliquée et comme entrepôt décisionnel, avec deux schémas séparés.

2.1.3 Scénarios d'organisation des données étudiés

Trois scénarios d'architecture ont été comparés avant de retenir le modèle 3 zones.

- Data Lake pur : très flexible mais sans modèle décisionnel structuré, exigeant un écosystème Hadoop complet pour des volumes qui ne le justifient pas.
- Data Warehouse classique unique : simple mais peu adapté à l'ingestion de fichiers plats hétérogènes et à la traçabilité RGPD des transformations.
- Architecture hybride en médaillon : équilibre entre souplesse d'ingestion en Bronze, qualité et conformité en Silver, et performance analytique en Gold.

Choix stratégique

Le scénario hybride en médaillon a été retenu car il permet de gérer proprement l'hétérogénéité des sources, de tracer les transformations et de préserver un modèle décisionnel robuste, sans surdimensionner l'infrastructure.

2.2 Description des composants techniques conteneurisés

L'architecture technique du projet est intégralement conteneurisée. Chaque zone fonctionnelle est associée à un ou plusieurs conteneurs, interconnectés via des réseaux Docker distincts.

2.2.1 Zone Bronze : stockage brut des sources

La zone Bronze se compose des volumes et services suivants :

- Un conteneur PostgreSQL exposant le schéma **operational**, qui réplique la base de soins du SI hospitalier.
- Des volumes locaux montés pour stocker les dumps PostgreSQL, les fichiers CSV FINESS et les flux plats de satisfaction et de décès.
- Un réseau Docker interne réservé à la Bronze, non accessible directement depuis la couche de restitution.

Cette zone ne réalise aucune transformation. Elle garantit la conservation des sources brutes et la possibilité de rejouer les flux au besoin.

2.2.2 Zone Silver : traitement, nettoyage et orchestration

La zone Silver est le coeur des transformations ETL :

- Un conteneur d'exécution des scripts Python qui s'appuie sur **pandas** et **SQLAlchemy** pour charger les données de Bronze, appliquer les règles métier et pousser les résultats vers Gold.
- Un conteneur Apache Airflow (optionnel mais prévu dans l'architecture) chargé d'orchestrer les pipelines sous forme de DAGs, de planifier les exécutions et de tracer les dépendances entre tâches.
- Un réseau Docker Silver qui relie ces conteneurs aux volumes de travail intermédiaires et à la base PostgreSQL.

Les principes détaillés de développement des scripts ETL sont présentés au chapitre 4.

2.2.3 Zone Gold : stockage analytique (OLAP)

La zone Gold est portée par un schéma dédié au sein de la même instance PostgreSQL :

- Schéma **datawarehouse** pour les tables de faits et de dimensions du modèle en constellation.
- Index adaptés aux usages analytiques, et séparation claire des droits d'accès par rapport au schéma **operational**.

Ce schéma est la seule zone de la base visible depuis l'outil de BI.

2.2.4 Couche restitution (exposition)

La couche de restitution repose sur Metabase :

- Un conteneur Metabase connecté en lecture seule au schéma **datawarehouse**.
- Aucun accès au schéma **operational** ni aux volumes Bronze, conformément aux exigences de cloisonnement et de confidentialité.

Metabase permet aux utilisateurs métier d'explorer les données, de construire des requêtes et de composer des tableaux de bord sans exposer les données sensibles.

2.3 Flux de données et cinématique des traitements

2.3.1 Enchaînement des étapes d'alimentation

Le flux de bout en bout se déroule selon la séquence suivante :

1. Ingestion des sources brutes dans les volumes Docker et dans le schéma **operational** (zone Bronze).
2. Exécution des scripts Python orchestrés par Airflow pour le nettoyage, l'enrichissement et la pseudonymisation (zone Silver).
3. Chargement des tables de faits et de dimensions dans le schéma **datawarehouse** (zone Gold).
4. Interrogation du modèle décisionnel par Metabase pour produire des tableaux de bord et des rapports.

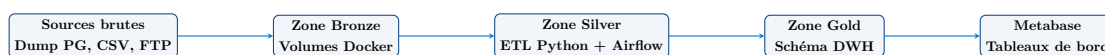


Figure 2.4 – Parcours de la donnée de la source jusqu'à la restitution

2.3.2 Stratégie d'actualisation et points de contrôle

La stratégie d'actualisation combine rechargements batch réguliers et contrôles systématiques :

- Une fréquence d'exécution (quotidienne ou hebdomadaire selon les flux) est définie dans Airflow pour chaque DAG.
- Les jobs vérifient les volumes traités et la cohérence des clés avant de valider le chargement en Gold.
- Les anomalies (valeurs hors plage, référentiels inconnus, échecs de pseudonymisation) sont journalisées pour correction.

Des points de contrôle sont positionnés à la fin de chaque zone afin de garantir que seules des données conformes et complètes puissent alimenter la zone suivante.

2.4 Schéma d'architecture technique

Les figures suivantes synthétisent l'ensemble des conteneurs, réseaux et flux de données de l'architecture Docker du projet.

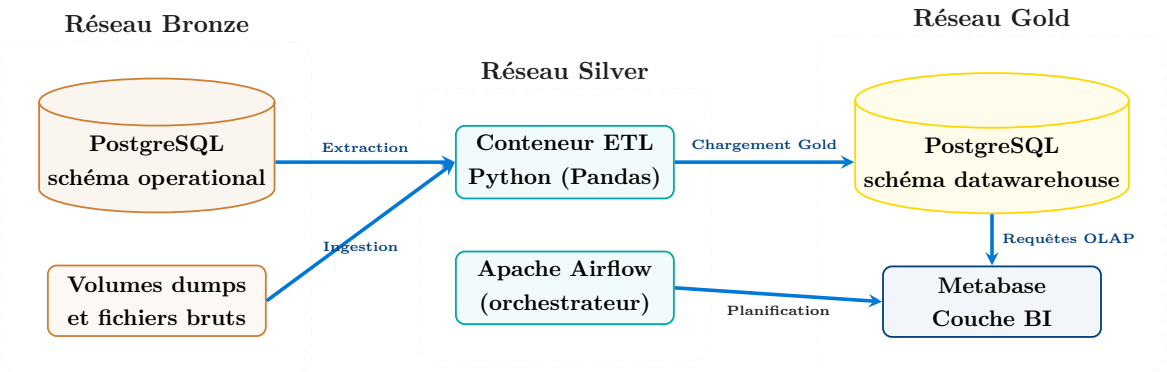


Figure 2.5 – Architecture réseau isolée et cinématique des conteneurs Docker CHU

La légende adoptée est la suivante :

- Cylindres tridimensionnels : conteneurs de stockage de données relationnelles et décisionnelles (schémas PostgreSQL).
- Encadrés pleins : conteneurs de traitement applicatif, d'orchestration et de restitution visuelle (flux Python, Airflow, Metabase).
- Encadrés pointillés : réseaux virtuels isolés au sein du moteur Docker (briques Bronze, Silver, Gold).
- Flèches directionnelles : flux de transfert logiques et cinématique des flux de données.

2.5 Synthèse de l'architecture retenue

2.5.1 Forces, limites et scalabilité de la solution

Forces	Limites
Stack open source légère et standardisée (Docker, Python, PostgreSQL, Metabase).	Volumétrie supportée limitée par la capacité mémoire des conteneurs et de pandas.
Séparation nette des zones Bronze, Silver, Gold et cloisonnement réseau.	Architecture orientée batch, sans traitement temps réel strict.
Traçabilité fine des transformations et alignement avec les exigences RGPD.	Montée en charge au delà de certains volumes nécessitant une évolution vers Spark.

Table 2.1 – Forces et limites de l'architecture actuelle

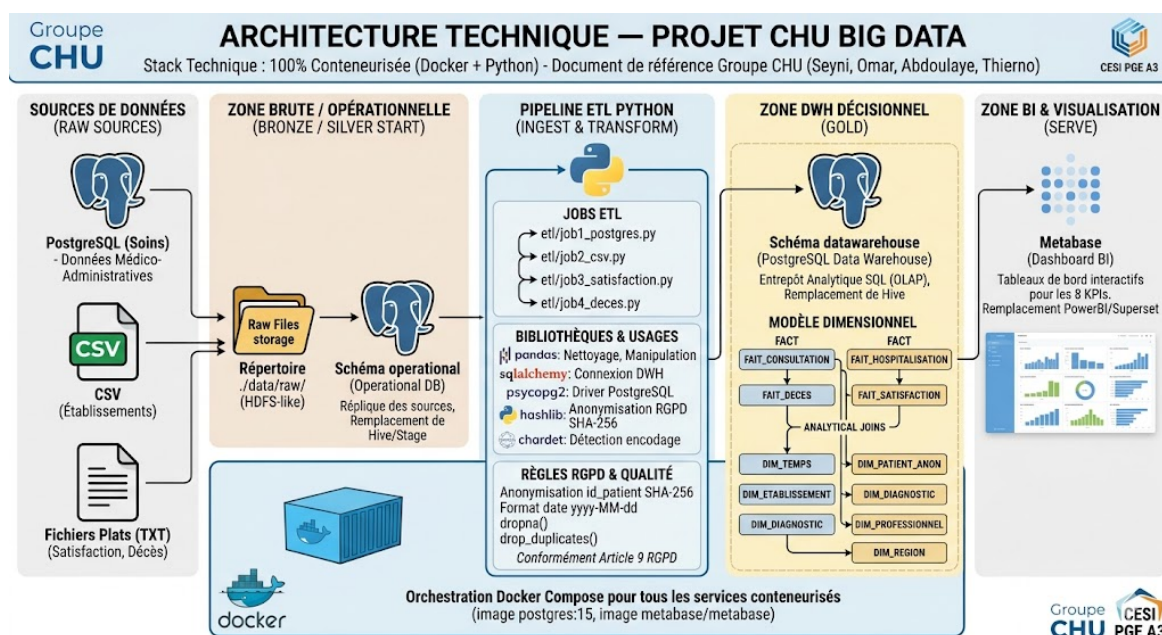


Figure 2.6 – Architecture résumée

L'architecture est suffisante pour un périmètre régional ou interrégional, avec des volumes de données compatibles avec une exécution quotidienne des traitements batch.

2.5.2 Évolutivité de l'architecture

L'architecture a été pensée pour rester évolutive :

- **Portabilité du code** : Les scripts ETL sont structurés de manière à pouvoir être portés vers PySpark sans modifier la structure globale des conteneurs.
- **Pérennité du réseau** : Les réseaux Docker et le découpage des zones restent valides en cas de remplacement du moteur de calcul au sein de la Silver.
- **Stabilité du modèle** : Le modèle décisionnel en Gold demeure inchangé, même si la technologie utilisée pour alimenter les tables de faits et de dimensions évolue.

En cas d'augmentation massive de la volumétrie, une transition vers une exécution distribuée (Spark) pourra être opérée de manière progressive, en conservant la même organisation en trois zones et la même interface BI côté utilisateurs.

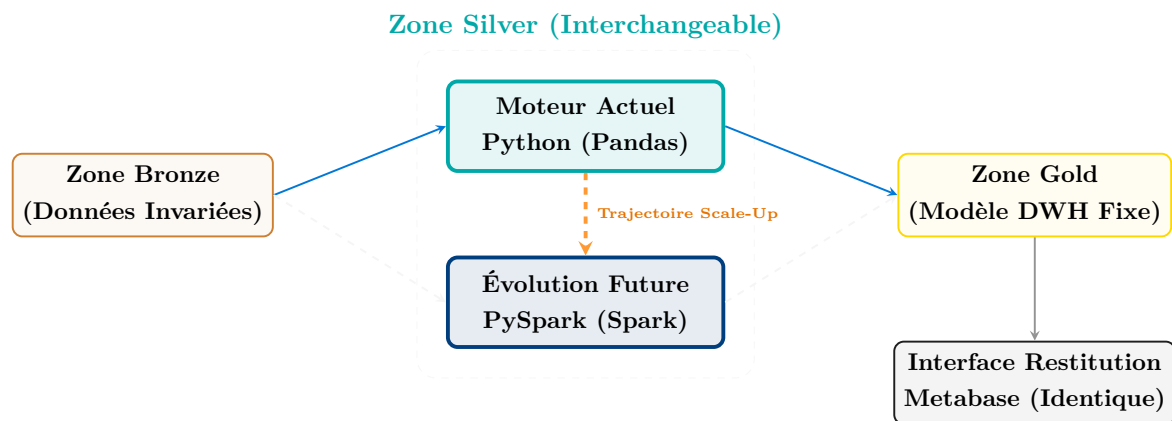


Figure 2.7 – Modularité de l'architecture et trajectoire d'évolution vers le calcul distribué

3. Modélisation dimensionnelle (Zone Gold)

La zone Gold concentre l'ensemble des structures décisionnelles utilisées par les tableaux de bord Metabase. Ce chapitre formalise les besoins analytiques, le choix du modèle décisionnel et la spécification détaillée des tables de faits et de dimensions.

3.1 Expression des besoins analytiques et indicateurs métiers

La conception de l'entrepôt de données du CHU repose sur une distinction claire entre deux types de métriques afin de garantir la flexibilité et l'évolutivité de la plateforme :

- **Mesures brutes** : valeurs numériques absolues et additives, stockées physiquement dans les tables de faits. Elles constituent le socle de calcul.
- **Mesures calculées** : indicateurs dérivés (taux, ratios, pourcentages) calculés dynamiquement dans la couche de restitution. Elles ne sont jamais stockées en dur pour éviter les biais lors de changements d'axes ou d'agrégations.

3.1.1 Indicateurs liés aux consultations et hospitalisations

Ce premier domaine couvre le suivi de l'activité de soins courante, ambulatoire et hospitalière.

- **FAIT_CONSULTATION (mesures brutes)** : nombre brut de consultations (via un *COUNT* de l'identifiant unitaire) et durée de l'acte calculée en minutes dans le flux ETL (différence entre l'heure de fin et l'heure de début).
- **FAIT_HOSPITALISATION (mesures brutes)** : volume absolu de séjours et durée globale d'hospitalisation exprimée en jours (*Jour_Hospitalisation*).

Ces mesures permettent de calculer ensuite des indicateurs tels que le taux d'occupation des lits, la durée moyenne de séjour ou les volumes d'activité par spécialité.

3.1.2 Indicateurs liés aux décès

Ce domaine est destiné au pilotage épidémiologique et à la surveillance de la mortalité à des fins de santé publique.

- **FAIT_DECES (mesures brutes)** : dénombrement absolu des événements de décès enregistrés sur l'exercice de référence, ventilés par région, tranche d'âge, sexe et éventuellement pathologie.

3.1.3 Indicateurs liés à la satisfaction des usagers

Ce dernier domaine agrège l'expérience patient issue des enquêtes nationales de la Haute Autorité de Santé (campagnes e-Satis, indicateurs IQSS).

- **FAIT_SATISFACTION (mesures brutes)** : score de satisfaction globale, notes par dimension (accueil, prise en charge médicale, prise en charge infirmière, chambre, repas, organisation de la sortie), taux de recommandation brut, classement de l'établissement et volume de répondants.

3.1.4 Synthèse des mesures calculées (couche restitution)

Les taux et agrégats demandés par la direction sont calculés à la volée dans la couche BI à partir des mesures brutes. Le tableau suivant illustre quelques indicateurs clés.

Besoin analytique	Formule et tables mobilisées
Taux de consultation par établissement	$\text{COUNT}(\text{consult.}) / \text{COUNT}(\text{total}) \times 100$ <i>FAIT_CONSULTATION + DIM_TEMPS + DIM_ETAB.</i>
Taux de consultation par diagnostic	$\text{COUNT}(\text{consult.}) / \text{COUNT}(\text{total}) \times 100$ <i>FAIT_CONSULTATION + DIM_DIAG.</i>
Taux d'hospitalisation par période	$\text{COUNT}(\text{hospi.}) / \text{COUNT}(\text{patients actifs}) \times 100$ <i>FAIT_HOSPIT. + DIM_TEMPS</i>
Taux d'hospitalisation par diagnostic	$\text{COUNT}(\text{hospi.}) / \text{COUNT}(\text{total}) \times 100$ <i>FAIT_HOSPIT. + DIM_DIAG.</i>
Taux d'hospitalisation par sexe	$\text{COUNT}(\text{hospi.}) / \text{COUNT}(\text{total}) \times 100$ <i>FAIT_HOSPIT. + DIM_PATIENT</i>
Taux d'hospitalisation par tranche d'âge	$\text{COUNT}(\text{hospi.}) / \text{COUNT}(\text{total}) \times 100$ <i>FAIT_HOSPIT. + DIM_PATIENT</i>
Taux de consultation par professionnel	$\text{COUNT}(\text{consult.}) / \text{COUNT}(\text{total}) \times 100$ <i>FAIT_CONSULTATION + DIM_PROF.</i>
Volume de décès par région (2019)	$\text{COUNT}(\text{décès}) \text{ GROUP BY région}$ <i>FAIT_DECES + DIM_TEMPS + DIM_GEO.</i>
Taux de satisfaction par région (2020)	$\text{AVG}(\text{score_global}) \text{ GROUP BY région}$ <i>FAIT_SATISFACTION + DIM_GEO.</i>

Table 3.1 – Mesures calculées dynamiquement dans la couche de restitution

3.2 Choix du modèle décisionnel

3.2.1 Justification d'un schéma en constellation

L'architecture décisionnelle du CHU retient un modèle en constellation, caractérisé par plusieurs tables de faits spécialisées partageant un ensemble commun de dimensions conformes. Ce choix est motivé par la coexistence de processus métiers distincts (consultations, hospitalisations, décès, satisfaction) issus de sources hétérogènes.

L'utilisation d'une simple table de faits centralisée aurait généré un volume important de valeurs nulles et complexifié l'analyse croisée. À l'inverse, l'approche en constellation garantit :

- **Une intégrité référentielle transverse** : Les dimensions partagées (comme DIM_TEMPS ou DIM_GEOGRAPHIE) assurent que les analyses restent cohérentes lorsqu'on croise les différentes tables de faits.
- **Une évolutivité naturelle** : L'ajout ultérieur d'un nouveau processus (ex. la logistique des médicaments) nécessitera simplement l'ajout d'une nouvelle table de faits reliée aux dimensions préexistantes.

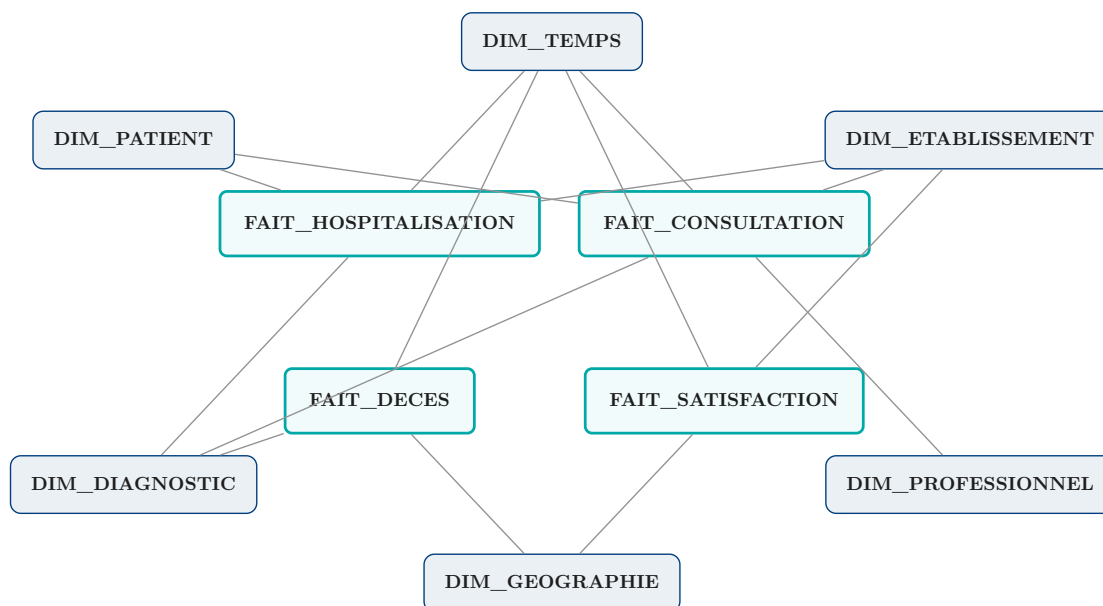


Figure 3.1 – Schéma en constellation simplifié final du modèle analytique CHU

Forcer l'ensemble des mesures dans une seule table de faits (schéma en étoile simple) aurait conduit à :

- une prolifération de colonnes non pertinentes ou nulles, notamment pour les faits liés aux décès et à la satisfaction ;
- des risques d'erreurs lors des agrégations, du fait des différences de granularité (événement patient unitaire vs score annuel d'établissement) ;

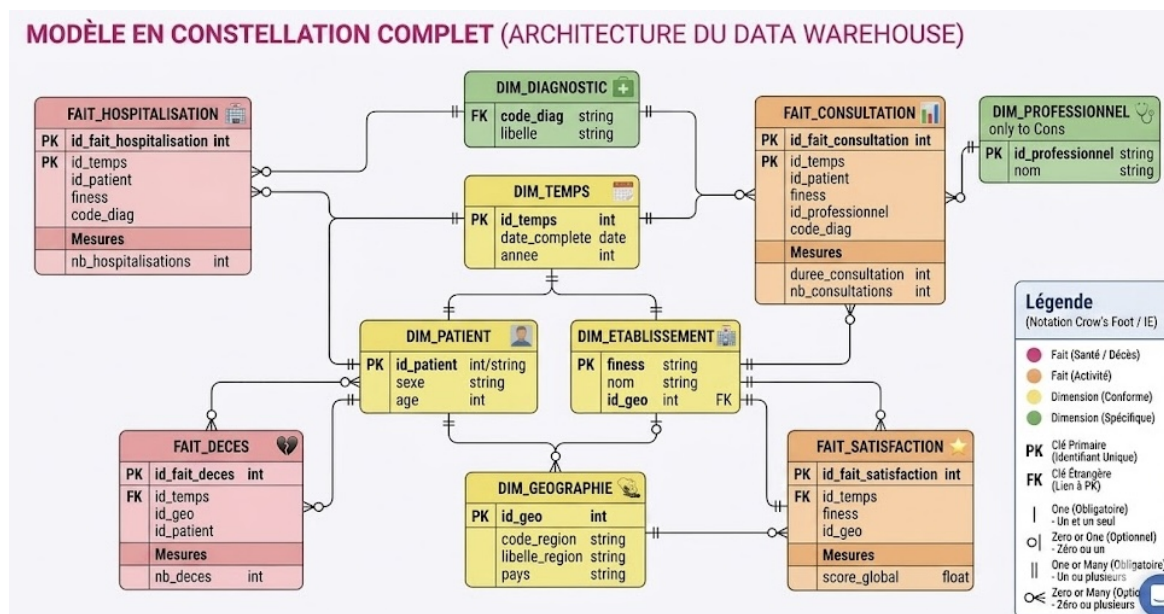


Figure 3.2 – Modèle en Constellation complet légendé

- une complexité accrue des règles de gestion pour garantir la cohérence des analyses.

La constellation permet au contraire d'isoler chaque coeur métier dans une table de faits dédiée, tout en partageant les dimensions communes (temps, patient, établissement, géographie).

3.2.2 Granularité des faits et niveaux d'agrégation

Chaque table de faits est définie à la granularité la plus fine compatible avec les sources :

- **FAIT_CONSULTATION** : une ligne par consultation unitaire (source : PostgreSQL opérationnel).
- **FAIT_HOSPITALISATION** : une ligne par séjour hospitalier complet (source : Hospitalisations.csv).
- **FAIT_DECES** : une ligne par événement de décès (source : `deces.csv`), avec une liaison vers *DIM_PATIENT* configurée en nullable du fait du rapprochement probabiliste.
- **FAIT_SATISFACTION** : une ligne par combinaison établissement / année / type d'enquête (source : Open Data HAS).

Cette granularité permet d'agréger ensuite les indicateurs au niveau souhaité (par établissement, par région, par période) sans perte d'information.

3.3 Spécification des tables de faits

3.3.1 Table de faits FAIT_HOSPITALISATION

Cette table de faits centralise les séjours hospitaliers en reliant chaque enregistrement aux dimensions temporelle, patient, établissement et diagnostic. Elle fournit les mesures de volumétrie nécessaires au suivi de l'activité d'hospitalisation.

Colonne	Type	Contrainte	Description
id_fait_hospitalisation	INT	PK	Clé primaire artificielle incrémentale
id_temps	INT	FK	Liaison vers <i>DIM_TEMPS</i>
id_patient	INT	FK	Liaison vers <i>DIM_PATIENT</i>
finess	VARCHAR	FK	Clé unifiée vers <i>DIM_ETABLISSEMENT</i>
code_diag	VARCHAR	FK	Liaison vers <i>DIM_DIAGNOSTIC</i>
duree_sejour	INT	Mesure	Équivalent de Jour_Hospitalisation
nb_hospitalisations	INT	Mesure	Valeur constante = 1 (facilite la sommation)

Table 3.2 – Structure de la table FAIT_HOSPITALISATION

Le diagramme en étoile associé met en évidence la position centrale de la table de faits par rapport aux dimensions temps, patient, établissement et diagnostic, ainsi que les mesures d'activité d'hospitalisation.

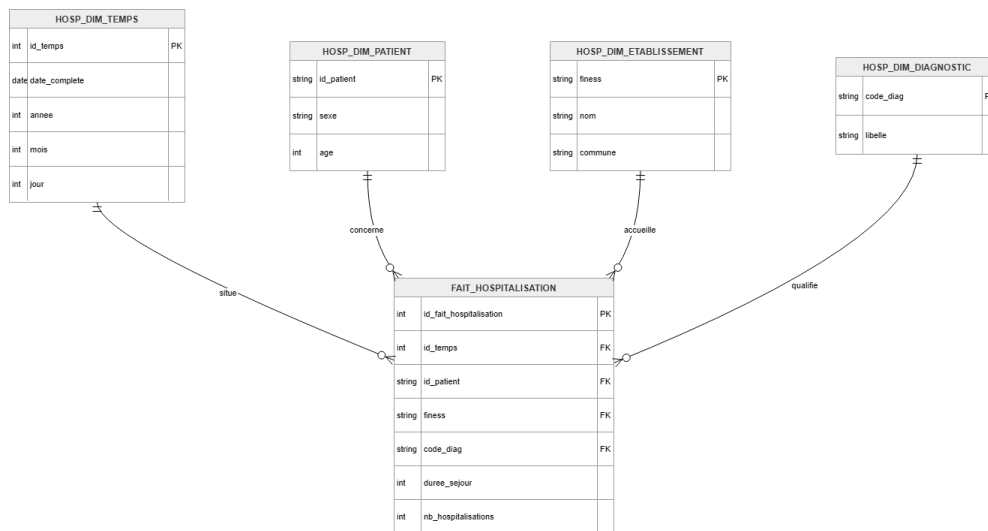


Figure 3.3 – Diagramme en étoile de la table FAIT_HOSPITALISATION

3.3.2 Table de faits FAIT_CONSULTATION

Cette table de faits décrit les consultations ambulatoires en reliant chaque acte aux dimensions temps, patient, professionnel, établissement et diagnostic. Elle permet de suivre la charge de travail et la durée des consultations.

Colonne	Type	Contrainte	Description
id_fait_consultation	INT	PK	Clé primaire artificielle incrémentale
id_temps	INT	FK	Liaison vers <i>DIM_TEMPS</i>
id_patient	INT	FK	Liaison vers <i>DIM_PATIENT</i>
id_professionnel	VARCHAR	FK	Liaison vers <i>DIM_PROFESSIONNEL</i>
code_diag	VARCHAR	FK	Liaison vers <i>DIM_DIAGNOSTIC</i>
finess	VARCHAR	FK	Clé unifiée vers <i>DIM_ETABLISSEMENT</i>
duree_consultation	INT	Mesure	Durée en minutes (Heure_fin – Heure_debut)
nb_consultations	INT	Mesure	Valeur constante = 1 (facilite la sommation)

Table 3.3 – Structure de la table FAIT_CONSULTATION

Le diagramme en étoile associé met en relation la table de faits avec les dimensions temps, patient, professionnel, établissement et diagnostic, afin de cartographier les principaux axes d'analyse des consultations.

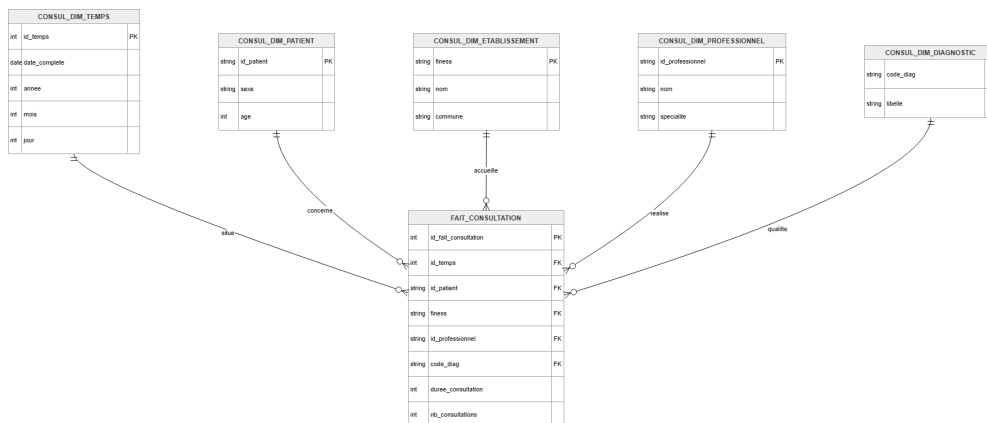


Figure 3.4 – Diagramme en étoile de la table FAIT_CONSULTATION

3.3.3 Table de faits FAIT_DECES

Cette table de faits recense les décès en les rattachant aux dimensions temporelle, géographique et, lorsque possible, patient. Elle sert de base aux analyses de mortalité par période, zone et population.

Colonne	Type	Contrainte	Description
id_fait_deces	INT	PK	Clé primaire artificielle incrémentale
id_temps	INT	FK	Liaison vers <i>DIM_TEMPS</i>
id_geo	INT	FK	Liaison vers <i>DIM_GEOGRAPHIE</i>
id_patient	INT	FK nullable	Liaison vers <i>DIM_PATIENT</i> (rapprochement probabiliste)
nb_deces	INT	Mesure	Valeur constante = 1 (facilite la sommation)

Table 3.4 – Structure de la table FAIT_DECES

Le diagramme en étoile met en lumière les liens entre la table de faits et les dimensions temps, géographie et patient, pour faciliter les analyses territoriales et temporelles de la mortalité.

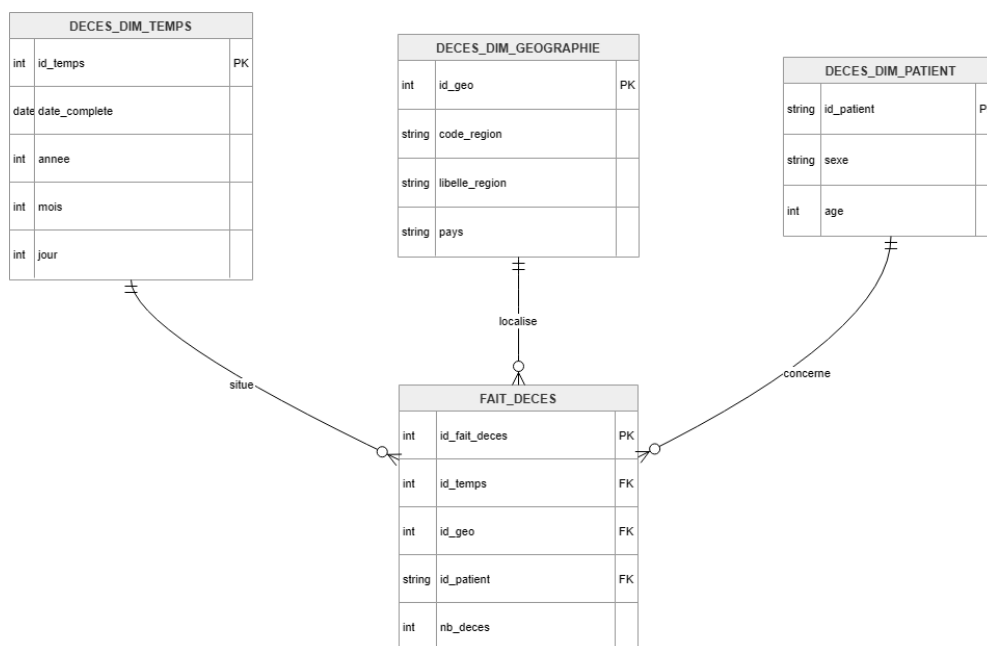


Figure 3.5 – Diagramme en étoile de la table FAIT_DECES

3.3.4 Table de faits FAIT_SATISFACTION

Cette table de faits agrège les résultats des enquêtes de satisfaction en les rattachant aux dimensions temps, établissement et géographie. Elle porte les différents scores thématiques ainsi que les indicateurs globaux de perception.

Colonne	Type	Contrainte	Description
id_fait_satisfaction	INT	PK	Clé primaire artificielle incrémentale
id_temps	INT	FK	Liaison vers <i>DIM_TEMPS</i>
finess	VARCHAR	FK	Clé unifiée vers <i>DIM_ETABLISSEMENT</i>
id_geo	INT	FK	Liaison vers <i>DIM_GEOGRAPHIE</i>
type_enquete	VARCHAR	Attribut	Nature de la campagne (48H ou CA)
score_global	FLOAT	Mesure	Note globale ajustée de l'établissement
score_accueil	FLOAT	Mesure	Score lié à l'accueil
score_pec_medical	FLOAT	Mesure	Score de prise en charge médicale
score_pec_infirmier	FLOAT	Mesure	Score de prise en charge infirmière
score_chambre	FLOAT	Mesure	Score lié au confort de la chambre
score_repas	FLOAT	Mesure	Score lié à la qualité des repas
score_sortie	FLOAT	Mesure	Score lié à l'organisation de la sortie
nb_repondants	INT	Mesure	Nombre de patients répondants
taux_recommandation	FLOAT	Mesure	Pourcentage brut de recommandation
classement	VARCHAR	Attribut	Label de performance (A, B, C, D, DI, NR)

Table 3.5 – Structure de la table FAIT_SATISFACTION

Le diagramme en étoile visualise la table de faits au centre des dimensions temps, établissement et géographie, en mettant en avant les différents scores de satisfaction et indicateurs dérivés.

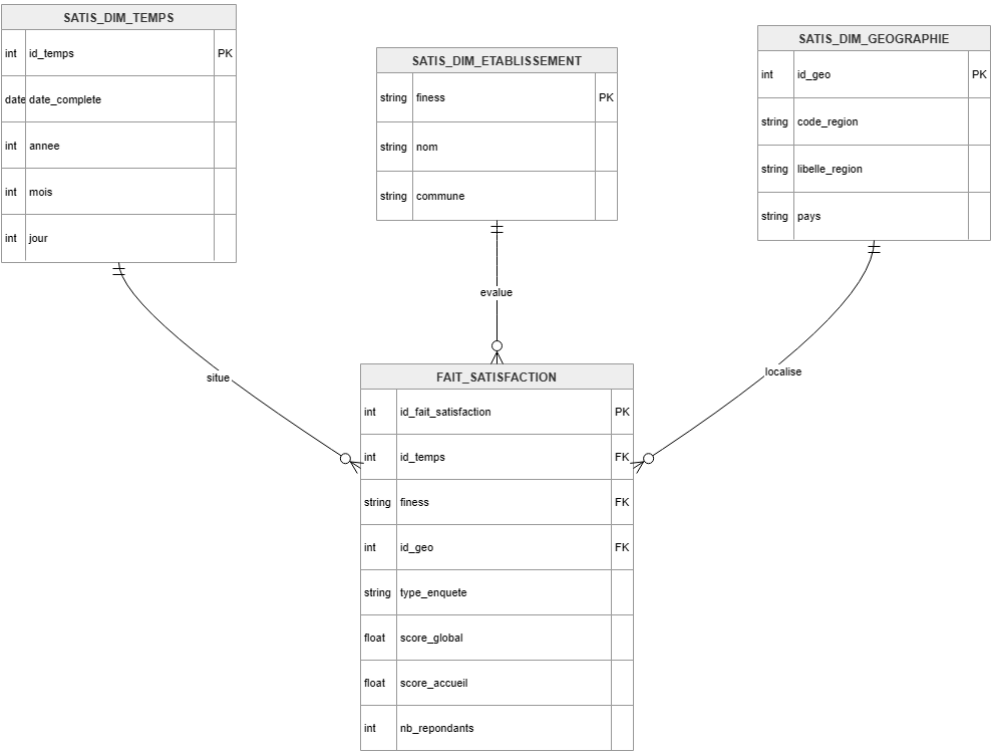


Figure 3.6 – Diagramme en étoile de la table FAIT_SATISFACTION

3.4 Spécification des tables de dimensions

Les dimensions fournissent les axes contextuels indispensables au filtrage et aux analyses croisées.

Colonne	Type	Rôle	Spécification
finess	VARCHAR	PK	Code FINESS géographique (clé unifiée)
nom	VARCHAR	Attribut	Raison sociale de la structure
commune	VARCHAR	Attribut	Ville d'implantation
code_postal	VARCHAR	Attribut	Code postal de la structure
code_commune	VARCHAR	Attribut	Code officiel INSEE de la commune
id_geo	INT	FK	Liaison obligatoire vers <i>DIM_GEOGRAPHIE</i>

Table 3.6 – Structure de la dimension DIM_ETABLISSEMENT

Colonne	Type	Rôle	Spécification
code_diag	VARCHAR	PK	Code alphanumérique CIM-10
libelle	VARCHAR	Attribut	Libellé textuel de la pathologie

Table 3.7 – Structure de la dimension DIM_DIAGNOSTIC

Colonne	Type	Rôle	Spécification
id_professionnel	VARCHAR	PK	Identifiant unifié (RPPS / ADELI)
nom	VARCHAR	Attribut	Nom de famille du professionnel
profession	VARCHAR	Attribut	Corps de métier (médecin, sage-femme, IDEL, ...)
specialite	VARCHAR	Attribut	Libellé complet de la spécialité clinique

Table 3.8 – Structure de la dimension DIM_PROFESSIONNEL

Colonne	Type	Rôle	Spécification
id_geo	INT	PK	Clé primaire technique artificielle
code_region	VARCHAR	Attribut	Code numérique de la région administrative
libelle_region	VARCHAR	Attribut	Nom de la région administrative française
pays	VARCHAR	Attribut	Identifiant national global

Table 3.9 – Structure de la dimension DIM_GEOGRAPHIE

3.5 Intégration, intégrité et couverture du modèle

3.5.1 Clés primaires, clés étrangères et intégrité référentielle

L'intégration en constellation repose sur un système cohérent de clés primaires (PK) et de clés étrangères (FK).

- **PK des dimensions** : chaque dimension possède une clé primaire unique (entier technique ou code métier stable comme le FINESS ou le code CIM-10).
- **PK des faits** : chaque table de faits possède une clé primaire propre (`id_fait_consultation`, `id_fait_hospitalisation`, etc.).
- **FK dans les faits** : les tables de faits centralisent les PK de toutes les dimensions auxquelles elles se rattachent, ce qui garantit l'intégrité des jointures.

Les relations sont systématiquement de type 1 : N : une ligne de dimension peut être liée à plusieurs lignes de faits, mais chaque fait ne pointe que vers une seule ligne de dimension.

3.5.2 Matrice de couverture des indicateurs métiers

La matrice suivante vérifie que chaque besoin exprimé dans le chapitre 1 dispose d'une traduction explicite dans le modèle décisionnel. :6]

Besoin utilisateur	Table de faits	Dimensions mobilisées
Évolution du volume de consultations ambulatoires	FAIT_CONSULTATION (nb_consultations)	<i>DIM_TEMPS</i> (année, mois), <i>DIM_ETAB.</i> (nom, commune)
Durée moyenne de séjour par pathologie	FAIT_HOSPITALISATION (duree_sejour)	<i>DIM_DIAG.</i> (libellé), <i>DIM_TEMPS</i>
Analyse de la mortalité par région	FAIT_DECES (nb_deces)	<i>DIM_GEO.</i> (région), <i>DIM_PATIENT</i> (tranche d'âge, sexe)
Score de satisfaction sur l'accueil	FAIT_SATISFACTION (score_accueil)	<i>DIM_ETAB.</i> (nom), <i>DIM_TEMPS</i> (campagne)

Table 3.10 – Couverture des indicateurs métiers par le modèle en constellation

Cette matrice valide que les huit indicateurs décisionnels définis au chapitre 1 peuvent être reconstruits de manière univoque à partir des tables de faits et des dimensions décrites dans ce chapitre, sans logique implicite cachée dans la couche de restitution.

4. Scripts ETL Python d'alimentation (Zone Silver)

Les pipelines ETL constituent le coeur opérationnel de l'architecture en médaille. Ils assurent la transition entre les données brutes de la zone Bronze et le schéma décisionnel de la zone Gold, en appliquant les règles de qualité, de normalisation et de conformité RGPD décrites aux chapitres précédents. Ce chapitre présente les principes généraux de développement retenus, puis détaille chacun des quatre jobs Python mis en oeuvre.

4.1 Principes généraux de développement des pipelines

4.1.1 Structure type d'un script ETL

Chaque pipeline suit une structure en trois phases classiques, systématiquement reproduite pour garantir la lisibilité et la maintenabilité du code :

1. **Extraction** : connexion à la source (PostgreSQL opérationnel ou fichier plat), lecture des données brutes dans un DataFrame pandas.
2. **Transformation** : application des règles métier, normalisation des formats, contrôles de qualité, pseudonymisation des identifiants sensibles.
3. **Chargement** : écriture dans le schéma `datawarehouse` via SQLAlchemy, avec gestion des conflits et des doublons.

```
1 from sqlalchemy import create_engine
2 import pandas as pd
3 from etl.utils import get_engine, anonymize, log_step
4
5 def run():
6     # 1. Extraction
7     engine_src = get_engine("operational")
8     df = pd.read_sql("SELECT * FROM table_source", engine_src)
9     log_step("Extraction", len(df))
10
11     # 2. Transformation
12     df = transform(df)
13     log_step("Transformation", len(df))
14
15     # 3. Chargement
16     engine_dwh = get_engine("datawarehouse")
```

```

17 df.to_sql("table_cible", engine_dwh,
18         if_exists="append", index=False)
19 log_step("Chargement", len(df))

```

Listing 4.1 – Structure type d'un pipeline ETL

4.1.2 Règles communes de transformation

Un ensemble de règles transversales est appliqué dans tous les pipelines, centralisées dans le module `etl/utils.py` :

- **Format de date** : conversion stricte au format YYYY-MM-DD via `pd.to_datetime()` avec `errors='coerce'`.
- **Valeurs manquantes** : suppression des lignes dont une clé étrangère est nulle via `dropna(subset=[...])`.
- **Doublons** : appel systématique à `drop_duplicates()` sur les colonnes constituant la clé fonctionnelle.
- **Encodage** : détection automatique via `chardet` pour les fichiers CSV d'origine hétérogène.

4.1.3 Module centralisé `etl/utils.py`

```

1 import hashlib, hmac, os, logging
2 from sqlalchemy import create_engine
3
4 SECRET_KEY = os.getenv("HASH_SECRET_KEY", "")
5
6 def get_engine(schema: str):
7     user = os.getenv("DB_USER")
8     pwd = os.getenv("DB_PASSWORD")
9     host = os.getenv("DB_HOST", "localhost")
10    port = os.getenv("DB_PORT", "5432")
11    db = os.getenv("DB_NAME")
12    url = f"postgresql+psycopg2://{user}:{pwd}@{host}:{port}/{db}"
13    return create_engine(
14        url, connect_args={"options": f"-csearch_path={schema}"}
15    )
16
17 def anonymize(value: str) -> str:
18     """Pseudonymise un identifiant par HMAC-SHA256 tronque a 16
19        cars."""
20    digest = hmac.new(
21        SECRET_KEY.encode(), str(value).encode(), hashlib.sha256
22    ).hexdigest()
23    return digest[:16]

```

```

23
24 def log_step(step: str, count: int) -> None:
25     logging.info(f"[ETL] {step} : {count} lignes traitees.")

```

Listing 4.2 – Module utilitaire centralisé etl/utils.py

4.2 Pipeline d'alimentation des données médico-administratives

4.2.1 Périmètre et source

Ce premier job extrait les activités cliniques depuis le schéma `operational` de PostgreSQL et alimente les tables `FAIT_CONSULTATION` et `FAIT_HOSPITALISATION`.

Paramètre	Valeur
Source	Schéma <code>operational</code> (PostgreSQL)
Tables lues	<code>consultations</code> , <code>hospitalisations</code> , <code>patients</code>
Tables alimentées	<code>FAIT_CONSULTATION</code> , <code>FAIT_HOSPITALISATION</code>
Fréquence	Quotidienne (DAG Airflow à 02h00)
Mode de chargement	Incrémental sur <code>date_acte</code>

Table 4.1 – Paramètres du Job 1 – Activités cliniques

4.2.2 Logique de transformation

- Pseudonymisation de `id_patient` et `id_medecin` via `anonymize()`.
- Suppression des colonnes nominatives (`nom`, `prenom`, `adresse`, `num_secu`).
- Calcul de la durée de consultation en minutes (`Heure_fin` – `Heure_debut`).
- Calcul de la durée de séjour en jours (`Date_sortie` – `Date_entree`).
- Jointure avec `DIM_TEMPS` sur la date normalisée pour résoudre `id_temps`.

```

1 from etl.utils import get_engine, anonymize, log_step
2 import pandas as pd
3
4 def run():
5     engine_src = get_engine("operational")
6     engine_dwh = get_engine("datawarehouse")
7
8     df = pd.read_sql("SELECT * FROM consultations", engine_src)
9     log_step("Extraction", len(df))
10
11     # Pseudonymisation
12     df["id_patient"] = df["id_patient"].apply(anonymize)
13     df["id_medecin"] = df["id_medecin"].apply(anonymize)
14

```

```

15  # Suppression colonnes nominatives
16  df.drop(columns=["nom", "prenom", "adresse", "num_secu"],
17          inplace=True, errors="ignore")
18
19  # Calcul duree consultation
20  df["duree_consultation"] = (
21      pd.to_datetime(df["heure_fin"])
22      - pd.to_datetime(df["heure_debut"])
23  ).dt.total_seconds() // 60
24
25  # Nettoyage
26  df.dropna(subset=["id_patient", "code_diag", "finess"],
27          inplace=True)
28  df.drop_duplicates(inplace=True)
29
30  df.to_sql("FAIT_CONSULTATION", engine_dwh,
31          if_exists="append", index=False)
32  log_step("Chargement FAIT_CONSULTATION", len(df))

```

Listing 4.3 – Job 1 – pseudonymisation et nettoyage des données cliniques

4.3 Pipeline d'intégration du référentiel des établissements

4.3.1 Périmètre et source

Le Job 2 traite le fichier CSV national du référentiel FINESS et alimente DIM_ETABLISSEMENT ainsi que DIM_GEOGRAPHIE.

Paramètre	Valeur
Source	Fichier <code>etablissements.csv</code> (volume Bronze)
Tables alimentées	DIM_ETABLISSEMENT, DIM_GEOGRAPHIE
Fréquence	Hebdomadaire (lundi à 01h00)
Mode de chargement	Rechargement complet (<code>replace</code>)

Table 4.2 – Paramètres du Job 2 – Référentiel des établissements

4.3.2 Logique de transformation

- Détection automatique de l'encodage via `chardet.detect()`.
- Nettoyage des codes FINESS : suppression des espaces, rejet des codes de longueur incorrecte ($\neq 9$ caractères).
- Normalisation textuelle des noms d'établissements et de communes.
- Construction de DIM_GEOGRAPHIE par dédoublement sur `code_region`, puis résolution de `id_geo` par jointure.

```

1 import chardet, pandas as pd
2 from etl.utils import get_engine, log_step
3
4 def detect_encoding(filepath: str) -> str:
5     with open(filepath, "rb") as f:
6         result = chardet.detect(f.read(100_000))
7     return result["encoding"]
8
9 def run(filepath: str):
10     engine_dwh = get_engine("datawarehouse")
11
12     enc = detect_encoding(filepath)
13     df = pd.read_csv(filepath, encoding=enc, sep=";")
14     log_step("Lecture FINESS", len(df))
15
16     df["fitness"] = df["fitness"].str.strip().str.upper()
17     df = df[df["fitness"].str.len() == 9]
18
19     # Construction DIM_GEOGRAPHIE
20     df_geo = (df[["code_region", "libelle_region"]]
21              .drop_duplicates()
22              .reset_index(drop=True))
23     df_geo["id_geo"] = df_geo.index + 1
24     df_geo.to_sql("DIM_GEOGRAPHIE", engine_dwh,
25                  if_exists="replace", index=False)
26
27     # Chargement DIM_ETABLISSEMENT
28     df = df.merge(df_geo[["code_region", "id_geo"]],
29                  on="code_region", how="left")
30     cols = ["fitness", "nom", "commune",
31            "code_postal", "code_commune", "id_geo"]
32     df[cols].to_sql("DIM_ETABLISSEMENT", engine_dwh,
33                    if_exists="replace", index=False)
34     log_step("Chargement DIM_ETABLISSEMENT", len(df))

```

Listing 4.4 – Job 2 – détection d’encodage et nettoyage FINESS

4.4 Pipeline de traitement des données de satisfaction

4.4.1 Périmètre et source

Le Job 3 traite les fichiers plats issus des campagnes e-Satis de la Haute Autorité de Santé (exercice 2020) et alimente FAIT_SATISFACTION.

4.4.2 Logique de transformation

- Lecture et parsing des fichiers plats FTP, avec détection d’encodage.

Paramètre	Valeur
Source	Fichiers plats FTP (campagnes 48H et CA, HAS 2020)
Tables alimentées	FAIT_SATISFACTION
Fréquence	Annuelle (chargement initial, puis mise à jour campagne)
Mode de chargement	Rechargement complet (replace)

Table 4.3 – Paramètres du Job 3 – Satisfaction

- Extraction et conversion des scores textuels en valeurs numériques (**NUMERIC**).
- Suppression des commentaires libres et verbatims pour éviter tout risque de réidentification indirecte.
- Calcul de la moyenne des scores par établissement et par dimension d'évaluation.
- Résolution des clés étrangères `finess`, `id_temps` et `id_geo` par jointure avec les dimensions.

```

1 import pandas as pd
2 from etl.utils import get_engine, detect_encoding, log_step
3
4 def run(filepath: str):
5     engine_dwh = get_engine("datawarehouse")
6
7     enc = detect_encoding(filepath)
8     df = pd.read_csv(filepath, encoding=enc, sep=";")
9     log_step("Lecture satisfaction", len(df))
10
11     # Suppression des verbatims (conformite RGPD)
12     cols_verbatim = [c for c in df.columns
13                     if "commentaire" in c.lower()
14                     or "verbatim" in c.lower()]
15     df.drop(columns=cols_verbatim, inplace=True, errors="ignore")
16
17     # Conversion des scores en numerique
18     score_cols = ["score_global", "score_accueil",
19                  "score_pec_medical", "score_pec_infirmier",
20                  "score_chambre", "score_repas", "score_sortie"]
21
22     for col in score_cols:
23         if col in df.columns:
24             df[col] = pd.to_numeric(
25                 df[col].str.replace(",", "."), errors="coerce"
26             )
27
28     # Nettoyage et deduplication

```

```

28 df["fitness"] = df["fitness"].str.strip().str.upper()
29 df.dropna(subset=["fitness", "score_global"], inplace=True)
30 df.drop_duplicates(
31     subset=["fitness", "annee", "type_enquete"], inplace=True
32 )
33
34 # Chargement
35 df.to_sql("FAIT_SATISFACTION", engine_dwh,
36         if_exists="replace", index=False)
37 log_step("Chargement FAIT_SATISFACTION", len(df))

```

Listing 4.5 – Job 3 – traitement des fichiers de satisfaction

4.5 Pipeline de traitement des registres de décès

4.5.1 Périmètre et source

Le Job 4 traite les données épidémiologiques régionales issues du registre national des décès (fichier `deces.csv`, exercice 2019) et alimente la table `FAIT_DECES`.

Paramètre	Valeur
Source	Fichier <code>deces.csv</code> (volume Bronze, INSEE 2019)
Tables alimentées	<code>FAIT_DECES</code>
Fréquence	Annuelle (chargement initial puis mise à jour annuelle)
Mode de chargement	Rechargement complet (<code>replace</code>)

Table 4.4 – Paramètres du Job 4 – Registres de décès

4.5.2 Logique de transformation

- Importation et détection automatique de l'encodage du fichier source.
- Contrôles de cohérence chronologique : rejet des dates de décès antérieures à la date de naissance ou postérieures à la date courante de traitement.
- Agrégation des volumes à la maille région / année afin de limiter le risque de réidentification indirecte, conformément aux exigences RGPD.
- Résolution des clés étrangères `id_temps` et `id_geo` par jointure avec `DIM_TEMPS` et `DIM_GEOGRAPHIE`.
- Chargement en bloc dans `FAIT_DECES` avec la valeur constante `nb_deces = 1` par événement unitaire.

```

1 import pandas as pd
2 from etl.utils import get_engine, detect_encoding, log_step
3

```

```

4 def run(filepath: str):
5     engine_dwh = get_engine("datawarehouse")
6
7     enc = detect_encoding(filepath)
8     df = pd.read_csv(filepath, encoding=enc, sep=";")
9     log_step("Lecture deces", len(df))
10
11     # Normalisation des dates
12     df["date_deces"] = pd.to_datetime(
13         df["date_deces"], errors="coerce", format="%Y-%m-%d"
14     )
15     df["date_naissance"] = pd.to_datetime(
16         df["date_naissance"], errors="coerce", format="%Y-%m-%d"
17     )
18
19     # Controles de coherence chronologique
20     df = df[df["date_deces"] >= df["date_naissance"]]
21     df = df[df["date_deces"] <= pd.Timestamp.today()]
22     df.dropna(subset=["date_deces", "code_region"], inplace=True)
23
24     # Agregation region / annee (conformite RGPD)
25     df["annee"] = df["date_deces"].dt.year
26     df_agg = (
27         df.groupby(["code_region", "annee", "sexe", "tranche_age"
28             "])
29         .size()
30         .reset_index(name="nb_deces")
31     )
32
33     # Jointure DIM_GEOGRAPHIE
34     df_geo = pd.read_sql(
35         "SELECT id_geo, code_region FROM DIM_GEOGRAPHIE",
36         engine_dwh
37     )
38     df_agg = df_agg.merge(
39         df_geo, on="code_region", how="left"
40     )
41
42     # Jointure DIM_TEMPS
43     df_temps = pd.read_sql(
44         "SELECT id_temps, annee FROM DIM_TEMPS", engine_dwh
45     )
46     df_agg = df_agg.merge(df_temps, on="annee", how="left")
47
48     # Nettoyage final
49     df_agg.dropna(subset=["id_geo", "id_temps"], inplace=True)

```

```
48     df_agg.drop_duplicates(inplace=True)
49
50     # Chargement
51     df_agg.to_sql("FAIT_DECES", engine_dwh,
52                  if_exists="replace", index=False)
53     log_step("Chargement FAIT_DECES", len(df_agg))
```

Listing 4.6 – Job 4 – traitement et chargement des registres de décès

Synthèse des quatre jobs ETL

Les quatre pipelines couvrent l'intégralité des sources identifiées au chapitre 1 et alimentent toutes les tables de faits et dimensions du schéma décisionnel. Chaque job respecte les règles communes définies dans `etl/utils.py` : pseudonymisation HMAC-SHA256, suppression des colonnes nominatives, normalisation des dates et contrôle des doublons. L'orchestration par Apache Airflow garantit leur exécution ordonnée et leur traçabilité complète.

5. Conformité RGPD et sécurité des données

Travailler avec des données de santé impose des contraintes qui vont bien au-delà d'une bonne architecture technique. Chaque patient dont un diagnostic, une durée d'hospitalisation ou une date de décès transite dans notre pipeline dispose de droits garantis par le RGPD. Il est de la responsabilité du groupe CHU de les respecter de manière concrète et vérifiable.

Ce chapitre documente les choix mis en oeuvre pour assurer la conformité réglementaire. Il ne s'agit pas d'une liste de bonnes intentions, mais d'une description des mesures effectivement intégrées à l'architecture et aux jobs ETL de l'entrepôt décisionnel.

5.1 Cadre réglementaire et enjeux

5.1.1 L'article 9 du RGPD et nos sources de données

Les données manipulées dans ce projet (diagnostics CIM-10, causes de décès, durées d'hospitalisation, scores de satisfaction) relèvent de la catégorie des **données sensibles** définie à l'article 9 du RGPD. Leur traitement est en principe interdit, sauf à être couvert par une base légale explicite et documentée.

Dans le contexte du CHU, quatre grandes familles de sources sont concernées :

- **Base PostgreSQL opérationnelle** : données médico-administratives de production, issues du SI hospitalier.
- **Référentiel FINESS** : informations sur les établissements de santé.
- **Fichiers de satisfaction** : notes et verbatims issus des enquêtes e-Satis.
- **Registre des décès** : données agrégées issues des fichiers nationaux de mortalité.

Trois textes nationaux complètent le RGPD et s'imposent directement au projet :

- **Loi Informatique et Libertés (2018)** : précise les droits des patients et les obligations des établissements hospitaliers en matière de traitement de leurs données.
- **Certification HDS (Art. L.1111-8 CSP)** : rend obligatoire le recours à un hébergeur certifié pour tout stockage de données médicales personnelles. Cette certification conditionne le passage en production.

Source	Base légale	Exemples d'usages
PostgreSQL soins	Art. 9.2.h (soins)	Suivi des parcours de soins, pilotage médico-économique
Référentiel FINESS	Intérêt légitime / obligation légale	Cartographie des établissements, analyses par territoire
Fichiers e-Satis	Art. 9.2.h / 9.2.j	Indicateurs d'expérience patient, qualité perçue
Registre des décès	Art. 9.2.j (statistiques publiques)	Analyses épidémiologiques, mortalité par pathologie et région

Table 5.1 – Sources de données et bases légales associées

- **PGSSI-S (ANS)** : fixe les exigences minimales en matière d'authentification forte, de traçabilité des accès et de gestion stricte des habilitations.

5.1.2 Le risque de réidentification indirecte

Une idée reçue consiste à penser que supprimer le nom et le prénom d'un patient suffit à l'anonymiser. En pratique, la combinaison de quelques attributs en apparence anodins peut suffire à singulariser un individu. Ce risque a été formalisé par Latanya Sweeney via la notion de *k-anonymat*.

Dans un contexte hospitalier, certains croisements sont particulièrement sensibles :

- **Date de naissance précise + établissement + diagnostic rare** dans une commune de taille moyenne : ce triplet peut ne désigner qu'une seule personne.
- **Cause de décès spécifique + mois + commune** : la granularité des fichiers INSEE est suffisante pour isoler des cas individuels dans des zones rurales.
- **Commentaires libres de satisfaction** : un patient peut y mentionner le nom de son médecin ou la nature de son opération sans en mesurer les conséquences.

Risque identifié	Contre-mesure appliquée
Date de naissance précise + diagnostic rare	Remplacement par une tranche d'âge de 10 ans (ex. 40–49 ans)
Détails de décès très fins (date exacte, commune)	Agrégation à la maille région / année avant chargement dans l'EDW
Verbatims de satisfaction contenant des identifiants	Suppression des commentaires libres en zone Bronze, conservation de la seule note numérique
Croisements sur de petits effectifs (< 10 personnes)	Interdiction des requêtes analytiques trop fines, restriction à des groupes d'au moins 10 individus

Table 5.2 – Contre-mesures appliquées pour limiter la réidentification indirecte

5.2 Sécurité de l'infrastructure Docker

5.2.1 Isolation réseau par zone

Par défaut, tous les conteneurs Docker d'un même projet peuvent communiquer librement. Dès lors que l'on mélange des données brutes nominatives et des outils de visualisation accessibles aux utilisateurs finaux, ce fonctionnement est inacceptable.

Le fichier `docker-compose.yml` définit trois réseaux logiques distincts, alignés sur les zones Bronze, Silver et Gold :

Réseau	Type	Services rattachés
<code>net_bronze</code>	<code>internal</code>	PostgreSQL opérationnel, volumes de dumps bruts, collecteurs FTP
<code>net_silver</code>	Pont interne	Conteneur ETL Python, Apache Airflow, volumes de staging
<code>net_gold</code>	Pont exposé	Schéma <code>datawarehouse</code> PostgreSQL, Metabase

Table 5.3 – Réseaux Docker et services rattachés par zone

Le réseau `net_bronze` est déclaré avec l'option `internal: true` dans le `docker-compose.yml`. Aucun flux entrant ou sortant depuis l'hôte n'est possible, ce qui empêche tout utilisateur d'interroger directement les données brutes en dehors des jobs prévus. Metabase se trouve physiquement dans un réseau différent de PostgreSQL opérationnel et ne peut pas l'atteindre, même en cas de mauvaise configuration au niveau applicatif.

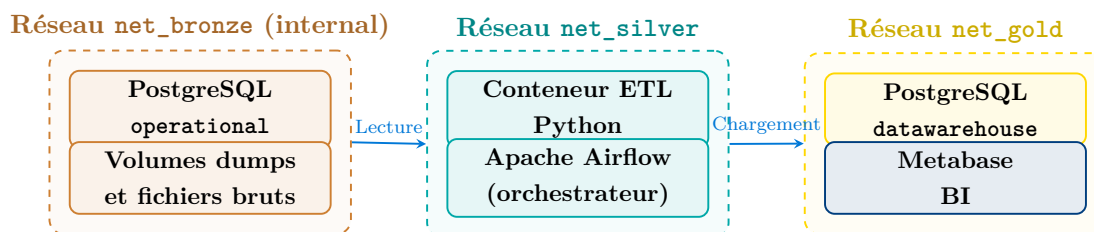


Figure 5.1 – Architecture Docker et flux inter-conteneurs

5.2.2 Séparation des privilèges en base de données

En complément du cloisonnement réseau, les droits en base sont segmentés par rôle. Trois rôles PostgreSQL ont été définis :

- **role_etl** : accès `SELECT` sur le schéma `operational` uniquement, utilisé par les scripts Python pour l'extraction nocturne.
- **role_metabase** : accès `SELECT` sur le seul schéma `datawarehouse`, via des vues ne retournant que des agrégats (`COUNT`, `AVG`).

- **role_admin** : accès complet réservé au DBA, protégé par authentification forte (MFA), jamais utilisé dans les traitements automatiques.

Concrètement, même si les identifiants Metabase étaient exfiltrés, un attaquant ne pourrait jamais accéder aux données nominatives ni aux tables opérationnelles.

5.3 Techniques de protection dans les jobs ETL

5.3.1 Pseudonymisation HMAC-SHA256 tronqué

Un simple hachage SHA-256 de l'identifiant patient ne suffit pas. Les numéros de séjour ou de sécurité sociale vivent dans un espace de valeurs limité et prévisible, ce qui rend une attaque par dictionnaire relativement simple.

Le projet CHU utilise donc un **HMAC-SHA256** avec les propriétés suivantes :

- Le hachage est calculé avec une clé secrète externe (**HASH_SECRET_KEY**).
- La sortie est tronquée à 16 caractères pour limiter la taille des colonnes.
- Sans la clé HMAC, il est pratiquement impossible de reconstruire l'identifiant d'origine.

```
1 import hashlib, hmac, os
2
3 SECRET_KEY = os.getenv("HASH_SECRET_KEY", "")
4
5 def anonymize(value: str) -> str:
6     """Pseudonymise un identifiant par HMAC-SHA256 tronque a 16
7         cars."""
8     digest = hmac.new(
9         SECRET_KEY.encode(), str(value).encode(), hashlib.sha256
10    ).hexdigest()
11    return digest[:16]
```

Listing 5.1 – Pseudonymisation HMAC-SHA256 dans le module ETL

Ce traitement est appliqué systématiquement à **id_patient**, **id_medecin** et **id_prescripteur** avant toute sortie de la zone Bronze vers la zone Silver. La clé **HASH_SECRET_KEY** fait l'objet d'une rotation tous les 90 jours.

5.3.2 Aucune donnée nominative au-delà de la zone Bronze

Il s'agit d'une règle de construction, pas d'une convention de bonne conduite. Les champs suivants ne sont jamais mappés dans les schémas cibles des jobs ETL :

- **nom**, **prenom**, **adresse_exacte**, **num_secu** pour les patients.
- Coordonnées personnelles des praticiens.

- Tout identifiant libre pouvant contenir des informations sensibles (commentaires, notes internes, verbatims).

Ces colonnes n’existent tout simplement pas dans les tables de l’entrepôt décisionnel. Un composant de contrôle vérifie à chaque exécution de pipeline que ces colonnes interdites sont absentes du schéma de sortie. En cas d’échec, le job s’arrête et une alerte est envoyée à l’administrateur ETL.

Contrôle automatique des colonnes interdites

Un test systématique est exécuté en fin de transformation pour vérifier l’absence de toute colonne nominative dans le schéma de sortie. Ce contrôle bloque le déploiement de tout pipeline non conforme et déclenche une alerte immédiate.

5.3.3 Gestion des secrets via fichiers `.env` et `coffre-fort`

Tous les identifiants de bases de données, la clé HMAC et les identifiants FTP sont externalisés dans un fichier `.env` exclu du dépôt Git. En environnement de développement, les variables sont chargées localement via Docker Compose.

En environnement de production, ce fichier est remplacé par une intégration HashiCorp Vault. Les secrets sont injectés dans les conteneurs au démarrage via le mécanisme de *Docker Secrets*, sans jamais transiter en clair sur le système hôte ni dans les variables d’environnement du système.

Secret	Stockage	Modalités
Identifiants PostgreSQL	Fichier <code>.env</code> / Vault	Injectés via Docker Compose en dev, Docker Secrets en prod
Clé HMAC (HASH_SECRET_KEY)	Fichier <code>.env</code> / Vault	Rotation obligatoire tous les 90 jours
Identifiants FTP	Fichier <code>.env</code> / Vault	Jamais en clair dans le code source ni dans les logs
Identifiants Metabase	Fichier <code>.env</code>	Compte dédié en lecture seule sur <code>datawarehouse</code> uniquement

Table 5.4 – Gestion des secrets par type et environnement

5.4 Gouvernance des données

5.4.1 Registre des traitements (Article 30)

L'article 30 du RGPD impose de documenter chaque traitement de données personnelles. Le tableau ci-dessous présente le registre minimal pour l'entrepôt de données du CHU.

Champ	Valeur
Responsable de traitement	Groupe CHU – Direction des Systèmes d'Information
DPO	À désigner avant mise en production (art. 37 – obligatoire pour un établissement de santé)
Finalité	Analyse décisionnelle : suivi des consultations, hospitalisations, décès et satisfaction
Base légale	Art. 9.2.h (soins), Art. 9.2.j (statistiques de santé publique)
Personnes concernées	Patients hospitalisés ou en consultation dans les établissements du groupe CHU
Données traitées	Données de santé pseudonymisées (diagnostics, séjours, décès, satisfaction)
Destinataires	Praticiens, chefs d'établissement, analystes autorisés
Conservation EDW	10 ans maximum (Art. R.1112-7 CSP)
Conservation ODS	30 jours glissants, puis purge automatique
Conservation logs	1 an (journaux d'accès et d'exécution des pipelines)
Transferts hors UE	Aucun
Mesures de sécurité	Chiffrement AES-256 au repos, TLS 1.3 en transit, RBAC, HMAC-SHA256, audit trail (pgaudit)

Table 5.5 – Registre synthétique des traitements (Article 30 RGPD)

5.4.2 Politique Git : données réelles interdites dans le dépôt

Les jeux de données utilisés en développement et en test sont strictement synthétiques, générés avec la bibliothèque **Faker**. Ils sont structurellement identiques aux données réelles mais ne font référence à aucun patient existant.

Trois mécanismes techniques renforcent cette politique :

- Le fichier `.gitignore` exclut systématiquement les fichiers `.env`, `.csv`, `.parquet`, `.json`, `.key` et `.pem`.
- Un hook `pre-commit` basé sur `gitleaks` bloque tout commit contenant des patterns caractéristiques de données sensibles (numéros de sécurité sociale, mots de passe, clés API).
- La branche `main` est protégée : tout job ETL touchant à la pseudonymisation ou aux schémas de données passe obligatoirement par une pull request avec

revue par un second membre de l'équipe avant merge.

5.4.3 Analyse d'impact (DPIA) et droits des patients

Une Analyse d'Impact relative à la Protection des Données (DPIA, article 35) est obligatoire avant la mise en production. Le projet remplit deux critères déclencheurs :

- Traitement à grande échelle de données de santé.
- Évaluation systématique de personnes (indicateurs de performance, scores de satisfaction).

La DPIA sera conduite en cinq étapes : description du traitement, démonstration de sa nécessité, analyse des risques, présentation des mesures prévues et validation par le DPO. En cas de risques résiduels élevés, une consultation préalable de la CNIL sera envisagée.

Sur les droits des patients (articles 15 à 22), les procédures techniques prévues sont les suivantes :

- **Droit d'accès** : la réidentification ponctuelle se fait via la clé HMAC, accessible uniquement au DPO ou à une personne dûment habilitée.
- **Rectification et effacement** : effectués par script ciblé sur l'identifiant pseudonymisé, avec propagation en cascade dans les tables de faits et dimensions concernées.
- **Opposition** : l'identifiant pseudonymisé est inscrit dans une table d'exclusion, filtrée systématiquement par toutes les requêtes analytiques.

5.4.4 Synthèse opérationnelle des mesures en place

Résumé des mesures de conformité effectivement implémentées

- Base légale documentée pour chaque source de données.
- Agrégation des données de décès à la maille région / année, suppression des verbatims de satisfaction dès la zone Bronze.
- Architecture conteneurisée sur trois réseaux Docker cloisonnés (`net_bronze`, `net_silver`, `net_gold`).
- Metabase sans accès physique ni logique aux données brutes ou au schéma `operational`.
- Pseudonymisation systématique par HMAC-SHA256 tronqué à 16 caractères, mur Bronze infranchissable par construction.
- Gestion des secrets via fichiers `.env` non versionnés et HashiCorp Vault en production, rotation des clés tous les 90 jours.
- Politique Git stricte : données réelles interdites, `gitleaks` en `pre-commit`, revue obligatoire des jobs ETL sensibles.
- Registre des traitements (article 30) tenu et DPIA à conduire avant passage en production.
- DPO à nommer avant mise en production (obligation légale pour tout établissement de santé).

6. Synthèse et perspectives

Ce premier livrable pose les fondations conceptuelles et techniques du système décisionnel du groupe CHU. Il formalise les choix d'architecture, documente le modèle dimensionnel, spécifie les pipelines ETL et statue sur la conformité réglementaire. Ce chapitre en dresse le bilan et ouvre sur les étapes suivantes.

6.1 Bilan du référentiel de données établi

Le passage d'une architecture lourde (Hadoop, Talend) à une stack moderne et conteneurisée (Docker, Python, PostgreSQL) dote le groupe CHU d'un environnement décisionnel performant, économique et respectueux des standards de confidentialité exigés dans le secteur de la santé.

Les éléments suivants ont été validés dans ce livrable :

- **Architecture** : un pattern en médaillon trois zones (Bronze, Silver, Gold) intégralement conteneurisé sous Docker, avec un cloisonnement réseau et une séparation des privilèges alignés sur les exigences RGPD.
- **Modèle décisionnel** : un schéma en constellation composé de quatre tables de faits spécialisées (FAIT_CONSULTATION, FAIT_HOSPITALISATION, FAIT_DECES, FAIT_SATISFACTION) et six dimensions conformes partagées.
- **Pipelines ETL** : quatre scripts Python modulaires couvrant l'intégralité des sources identifiées, avec pseudonymisation systématique, suppression des colonnes nominatives et contrôles de qualité automatisés.
- **Conformité RGPD** : base légale documentée pour chaque source, pseudonymisation HMAC-SHA256, registre des traitements tenu, DPIA planifiée avant mise en production.

Le modèle dimensionnel répond avec exactitude aux huit indicateurs décisionnels formulés au chapitre 1 : volumes de consultations, durées de séjour, taux de mortalité par région, scores de satisfaction par établissement.

6.2 Limites de la solution proposée

La solution retenue est adaptée au périmètre régional et interrégional du groupe CHU dans le cadre de ce livrable. Elle présente néanmoins des limites à prendre en compte pour les phases suivantes :

- **Volumétrie** : la bibliothèque `pandas` traite les données en mémoire vive. Au-delà de quelques dizaines de millions de lignes, les temps de traitement

Composant	Statut	Commentaire
Architecture médaillon	Validée	Trois zones Docker cloisonnées, IaC via <code>docker-compose.yml</code>
Modèle en constellation	Validé	Quatre tables de faits, six dimensions conformes
Pipelines ETL Python	Spécifiés	Quatre jobs couvrant toutes les sources Bronze
Conformité RGPD	En cours	Registre tenu, DPIA et DPO à finaliser avant production
Restitution Metabase	Prévue	Connexion en lecture seule au schéma <code>datawarehouse</code>

Table 6.1 – Tableau de bord de validation du Livrable 1

des jobs ETL augmentent significativement et peuvent dépasser les fenêtres batch disponibles.

- **Hébergement local** : l'architecture actuelle repose sur une infrastructure Docker locale. En environnement de production hospitalière, le recours à un hébergeur certifié HDS est obligatoire, ce qui implique une migration et un audit de conformité.
- **Traitement temps réel** : l'architecture est orientée batch. Elle ne couvre pas les besoins de flux en temps réel, par exemple pour des alertes cliniques ou des indicateurs de tension hospitalière en temps continu.
- **Gestion des SCD** : la gestion des *Slowly Changing Dimensions* de type 2 (historisation des changements structurels comme les fusions d'établissements) est prévue dans le code mais n'est pas encore implémentée dans les jobs ETL.

6.3 Transition vers les étapes suivantes

Ouverture vers le Livrable 2

Le prochain livrable s'attachera à l'optimisation physique et à la mise en exploitation de l'architecture définie dans ce document :

- Implémentation complète des scripts DDL dans PostgreSQL (création des schémas, contraintes, index).
- Stratégies de partitionnement temporel sur `DIM_TEMPS` et d'indexation B-Tree sur les clés étrangères des tables de faits.
- Mesure et optimisation des temps d'exécution des requêtes analytiques complexes (plans d'exécution, `EXPLAIN ANALYZE`).
- Configuration avancée des tableaux de bord Metabase et mise en place des droits d'accès par profil utilisateur.

- Implémentation des SCD de type 2 sur les dimensions DIM_ETABLISSEMENT et DIM_PROFESSIONNEL.

Glossaire

Architecture en médaillon

Pattern d'organisation des données en trois zones successives (Bronze, Silver, Gold), chacune correspondant à un niveau de maturité croissant de la donnée : ingestion brute, intégration et qualité, exposition analytique.

Bronze (zone) Première couche de l'architecture en médaillon. Elle stocke les données brutes telles qu'elles sont reçues des sources, sans transformation, afin de garantir la traçabilité et la possibilité de rejouer les traitements.

Bus architecture

Concept de Ralph Kimball désignant le partage de dimensions conformes entre plusieurs datamarts, assurant la cohérence des analyses croisées.

CIM-10 Classification Internationale des Maladies, 10^e révision. Nomenclature médicale internationale utilisée pour coder les diagnostics et causes de décès.

Constellation (schéma en)

Modèle décisionnel composé de plusieurs tables de faits autonomes partageant un ensemble commun de dimensions conformes. Adapté aux environnements où coexistent plusieurs processus métiers distincts.

DAG (Directed Acyclic Graph)

Graphe orienté acyclique utilisé par Apache Airflow pour représenter les dépendances entre les tâches d'un pipeline ETL.

Datamart Sous-ensemble thématique d'un entrepôt de données, orienté vers un domaine métier précis (hospitalisations, satisfaction, décès, etc.).

Dimension Table du modèle décisionnel fournissant les axes contextuels d'analyse (temps, patient, établissement, diagnostic, etc.). Les dimensions sont dénormalisées pour optimiser les performances des requêtes analytiques.

DPIA *Data Protection Impact Assessment* (Analyse d'Impact relative à la Protection des Données). Étude obligatoire avant

	tout traitement à grande échelle de données sensibles, définie à l'article 35 du RGPD.
DPO	<i>Data Protection Officer</i> (Délégué à la Protection des Données). Rôle obligatoire dans les établissements de santé, chargé de veiller au respect du RGPD.
EDW	<i>Enterprise Data Warehouse</i> . Entrepôt de données d'entreprise consolidant l'ensemble des sources de données d'une organisation à des fins d'analyse décisionnelle.
ETL	<i>Extract, Transform, Load</i> . Processus d'extraction des données depuis les sources, de transformation selon les règles métier, et de chargement dans l'entrepôt décisionnel.
FINESS	Fichier National des Établissements Sanitaires et Sociaux. Référentiel national identifiant chaque structure de soins par un numéro unique (code FINESS).
Gold (zone)	Troisième couche de l'architecture en médaillon. Elle héberge le modèle décisionnel structuré (tables de faits et dimensions) prêt à être interrogé par les outils de BI.
HDS	Hébergement de Données de Santé. Certification obligatoire en France pour tout prestataire hébergeant des données médicales personnelles (Art. L.1111-8 du Code de la Santé Publique).
HMAC-SHA256	<i>Hash-based Message Authentication Code</i> utilisant l'algorithme SHA-256. Technique de pseudonymisation combinant un hachage cryptographique avec une clé secrète externe, rendant le résultat irréversible sans cette clé.
IaC	<i>Infrastructure as Code</i> . Pratique consistant à décrire et provisionner l'infrastructure informatique via des fichiers de configuration versionnés (ex. <code>docker-compose.yml</code>).
k-anonymat	Propriété d'un jeu de données garantissant que chaque individu ne peut pas être distingué d'au moins $k - 1$ autres individus sur la base des attributs quasi-identifiants disponibles.
Kimball (modèle de)	Méthodologie de modélisation décisionnelle proposée par Ralph Kimball, reposant sur le schéma en étoile et les dimensions dénormalisées pour optimiser les analyses métier.
ODS	<i>Operational Data Store</i> . Zone de stockage intermédiaire contenant les données intégrées et nettoyées, servant de socle à

l'alimentation de l'entrepôt décisionnel.

OLAP

Online Analytical Processing. Traitement analytique en ligne permettant d'interroger rapidement de grands volumes de données selon plusieurs dimensions.

PGSSI-S

Politique Générale de Sécurité des Systèmes d'Information de Santé. Référentiel publié par l'ANS (Agence du Numérique en Santé) définissant les exigences minimales de sécurité pour les SI de santé.

Pseudonymisation

Technique de protection des données consistant à remplacer les identifiants directs par des identifiants de substitution, de telle sorte que la réidentification ne soit possible qu'avec une information complémentaire détenue séparément.

RBAC

Role-Based Access Control. Modèle de contrôle d'accès dans lequel les droits sont attribués à des rôles, et les utilisateurs héritent des droits du rôle qui leur est assigné.

RGPD

Règlement Général sur la Protection des Données. Texte fondateur du droit européen en matière de protection des données personnelles, entré en vigueur le 25 mai 2018.

SCD (Slowly Changing Dimension)

Dimension évoluant lentement dans le temps. Le type 2 consiste à conserver l'historique complet en créant une nouvelle ligne à chaque changement, avec des dates de validité.

Silver (zone)

Deuxième couche de l'architecture en médaillon. Elle réalise les transformations ETL : nettoyage, normalisation des référentiels, contrôles de qualité et pseudonymisation des données sensibles.

Surrogate key

Clé de substitution technique (entier auto-incrémenté) utilisée dans les dimensions du modèle décisionnel pour découpler l'entrepôt des identifiants opérationnels.

Table des figures

2.1	Principe du schéma en étoile pour l'activité médicale du CHU . . .	7
2.2	Architecture en médaillon Bronze, Silver, Gold pour le CHU . . .	8
2.3	Diagnostic et limites de l'architecture basée sur une Machine Virtuelle unique	9
2.4	Parcours de la donnée de la source jusqu'à la restitution	11
2.5	Architecture réseau isolée et cinématique des conteneurs Docker CHU	12
2.6	Architecture résumée	13
2.7	Modularité de l'architecture et trajectoire d'évolution vers le calcul distribué	14
3.1	Schéma en constellation simplifié final du modèle analytique CHU	17
3.2	Modèle en Constellation complet légendé	18
3.3	Diagramme en étoile de la table FAIT_HOSPITALISATION	20
3.4	Diagramme en étoile de la table FAIT_CONSULTATION	21
3.5	Diagramme en étoile de la table FAIT_DECES	22
3.6	Diagramme en étoile de la table FAIT_SATISFACTION	24
5.1	Architecture Docker et flux inter-conteneurs	40

Liste des tableaux

1.1	Indicateurs décisionnels du système CHU	5
1.2	Sources de données disponibles	6
2.1	Forces et limites de l'architecture actuelle	12
3.1	Mesures calculées dynamiquement dans la couche de restitution .	16
3.2	Structure de la table FAIT_HOSPITALISATION	20
3.3	Structure de la table FAIT_CONSULTATION	21
3.4	Structure de la table FAIT_DECES	22
3.5	Structure de la table FAIT_SATISFACTION	23
3.6	Structure de la dimension DIM_ETABLISSEMENT	25
3.7	Structure de la dimension DIM_DIAGNOSTIC	25
3.8	Structure de la dimension DIM_PROFESSIONNEL	25
3.9	Structure de la dimension DIM_GEOGRAPHIE	26
3.10	Couverture des indicateurs métiers par le modèle en constellation	27
4.1	Paramètres du Job 1 – Activités cliniques	30
4.2	Paramètres du Job 2 – Référentiel des établissements	31
4.3	Paramètres du Job 3 – Satisfaction	33
4.4	Paramètres du Job 4 – Registres de décès	34
5.1	Sources de données et bases légales associées	38
5.2	Contre-mesures appliquées pour limiter la réidentification indirecte	39
5.3	Réseaux Docker et services rattachés par zone	40
5.4	Gestion des secrets par type et environnement	42
5.5	Registre synthétique des traitements (Article 30 RGPD)	43
6.1	Tableau de bord de validation du Livrable 1	47
A.1	Mapping complet entre stack initiale et architecture retenue . . .	54
B.1	Inventaire des tables du schéma datawarehouse	55
C.1	Règles de qualité des données appliquées dans les pipelines ETL .	56

A. Mapping technologique détaillé

Le tableau suivant récapitule la correspondance complète entre les outils initialement prévus dans le cahier des charges et les composants effectivement retenus dans l'architecture Docker.

Outil initial	Équivalent retenu	Zone	Justification
HDFS	Volumes Docker	Bronze	Stockage brut léger, persistant et versionnable sans cluster distribué
Talend Open Studio	Scripts Python (pandas)	Silver	Flexibilité du code, traçabilité RGPD, versionnement Git natif
PostgreSQL opérationnel	PostgreSQL (operational)	Bronze	Réplication du SI hospitalier, extraction SQL directe
Hive	PostgreSQL (datawarehouse)	Gold	Performances OLAP suffisantes sur ce volume, sans cluster Hadoop
PowerBI / Superset	Metabase	BI	Légèreté, déploiement Docker, accès web sans licence propriétaire
Hadoop / MapReduce	Docker + Python	Silver	Consommation de ressources 20 à 60 fois inférieure à une VM Hadoop

Table A.1 – Mapping complet entre stack initiale et architecture retenue

B. Registre complet des tables du schéma datawarehouse

Le tableau suivant liste l'ensemble des tables du schéma décisionnel, leur type et leur rôle dans le modèle en constellation.

Nom de la table	Type	Rôle
FAIT_CONSULTATION	Table de faits	Mesures des actes ambulatoires (durée, volume, liens vers dimensions)
FAIT_HOSPITALISATION	Table de faits	Mesures des séjours hospitaliers (durée de séjour, volume)
FAIT_DECES	Table de faits	Dénombrement des décès par région, période, tranche d'âge et sexe
FAIT_SATISFACTION	Table de faits	Scores e-Satis par établissement, campagne et dimension d'évaluation
DIM_TEMPS	Dimension	Axe temporel (jour, mois, trimestre, année)
DIM_PATIENT	Dimension	Profil pseudonymisé des patients (sexe, tranche d'âge, commune)
DIM_ETABLISSEMENT	Dimension	Référentiel FINESS des structures de soins
DIM_DIAGNOSTIC	Dimension	Nomenclature CIM-10 des pathologies
DIM_PROFESSIONNEL	Dimension	Praticiens par spécialité (RPPS / ADELI)
DIM_GEOGRAPHIE	Dimension	Découpage régional pour les analyses macro

Table B.1 – Inventaire des tables du schéma datawarehouse

C. Politique de qualité des données

Les contrôles de qualité appliqués dans les pipelines ETL sont résumés dans le tableau ci-dessous.

Règle de qualité	Méthode Python	Application
Normalisation des dates	<code>pd.to_datetime(..., errors='coerce')</code>	Tous les pipelines, toutes les colonnes de type date
Suppression des doublons	<code>drop_duplicates()</code>	Avant tout chargement en Gold, sur la clé fonctionnelle
Valeurs nulles sur FK	<code>dropna(subset=[...])</code>	Rejet des lignes dont une clé étrangère est absente
Détection d'encodage	<code>chardet.detect()</code>	Fichiers CSV FINESSE, satisfaction, décès
Colonnes nominatives interdites	<code>drop(columns=[...])</code>	Job 1, avant tout chargement hors zone Bronze
Pseudonymisation	<code>anonymize()</code> (HMAC-SHA256)	<code>id_patient</code> , <code>id_medecin</code> , <code>id_prescripteur</code>
Cohérence chronologique	Filtrage conditionnel pandas	Job 4 : rejet des décès antérieurs à la naissance
Agrégation anti-réidentification	<code>groupby([région, année])</code>	Job 4 : aucun enregistrement individuel dans FAIT_DECES

Table C.1 – Règles de qualité des données appliquées dans les pipelines ETL